Full Stack Developer – Technical Assessment

Overview

As part of the second-round interview process, we'd like you to complete a practical development task based on the type of work you would be doing at Zephyr Insight, for our client Adclaro.

You'll be given a synthetic dataset representing marketing performance across different brands, markets and channels. Your task is to build a working application that allows users to explore the data, understand performance trends and investigate changes in key metrics.

You will have **5–10 days** to complete the task.

We're not expecting a finished product. We're more interested in how you approach the problem, the decisions you make and what you can deliver within the time available.

1. The task

Build an application that allows a user to explore marketing performance across different brands, markets, channels and time periods.

The dataset will include marketing investment, business outcomes and other factors that may influence performance.

You can decide how best to structure the application, which features to prioritise and how to implement them. Part of the task is making sensible decisions about what to build within the available time.

We'd like to see a working application rather than a collection of disconnected components.

2. Technical requirements

Your solution should demonstrate the following:

- **Python:** Implement the data processing, analytical calculations and application logic.

- **Data processing:** Load, validate and transform the supplied data, handling missing or inconsistent values where appropriate.
- **PostgreSQL:** Design a suitable database structure and use it to store and retrieve data.
- **Frontend:** Build an interface using Streamlit or another suitable framework.
- **Testing:** Include automated tests covering key parts of the application.
- **Version control:** Use Git to manage your code.
- **Documentation:** Explain how to set up and run the application and document the main technical decisions.

You have flexibility over the implementation. We're interested in seeing how you approach the problem, not whether you follow a prescribed architecture.

3. Suggested features

The following checklist is intended to help you decide what to build. You don't need to implement everything, and you're welcome to suggest features of your own.

Data processing and database

- Data ingestion and validation
- Handling missing, invalid or inconsistent data
- A repeatable data processing pipeline
- Appropriate PostgreSQL schema and database design
- Efficient queries and consideration of performance
- Clear separation of data processing and application logic

Application and visualisation

- Filters for dates, brands, markets or channels
- Headline KPIs, such as spend, revenue and sales volume
- Performance trends over time
- Comparisons across channels, brands or markets
- Clear visualisations and presentation of results
- Data export functionality

Analytics

- Comparison of performance between two periods

- Analysis of the factors contributing to changes in performance
- Calculation of efficiency metrics, such as return on ad spend (ROAS)
- Identification of trends or unusual movements
- Additional statistical analysis, where appropriate

Software quality

- Automated tests for key functionality
- Appropriate error handling and logging
- Secure handling of credentials and configuration
- Reproducible environment setup
- Automated quality checks or CI/CD

Cloud and production

- Proposed Azure deployment approach
- Authentication and access control considerations
- Monitoring and error reporting
- Consideration of performance and scalability
- Docker or other containerisation, where appropriate

4. What to submit

Please provide:

1. **The application:** A working version that we can run and review.
2. **Source code:** A Git repository containing your implementation.
3. **README:** Instructions for setting up and running the application, including database setup and how to run the tests.
4. **Priorities and decisions:** A short explanation of the features you chose to implement, what you left out and why.
5. **Next steps:** A brief outline of what you would improve or add with more time.

You don't need to deploy the application to Azure. However, please include a short explanation of how you would approach deploying and running it in Azure, including any services you would use and how you would manage security, monitoring and scaling.

5. Use of AI tools

You're welcome to use AI tools such as ChatGPT or GitHub Copilot during the task.

We expect you to review and test any generated code and understand the implementation you're submitting. You'll be asked to explain your approach and discuss parts of your code during the interview.

We'll be interested in how you've used AI as part of your development process, how you've checked its output and how you've dealt with any issues it introduced.

6. Second-round interview

After submitting your work, you'll be invited to present your solution and discuss it with the technical team.

The interview will cover:

- **Presentation:** A walkthrough of the application and the features you've implemented.
- **Technical discussion:** Your approach to the data, database design, architecture and key decisions.
- **Code walkthrough:** A closer look at selected parts of your implementation, including testing, maintainability and performance.
- **Live coding:** A short, related coding or debugging exercise lasting no more than 10 minutes.
    - Pre-requisites checklist for live coding task:
        - Python 3.9+ installed and working
        - A code editor or IDE (VS Code / PyCharm / similar)
        - Optional: a virtual environment if they prefer clean local setup
        - No third-party packages required for the exercise itself

The live exercise will use a standard task for all candidates. It is intended to assess your practical coding ability and how you approach a problem, rather than your ability to solve a complex problem under pressure.

7. What we're looking for

We'll be assessing your ability to:

- Build a working application using Python, SQL and an appropriate frontend framework
- Process and analyse data reliably
- Write clear, maintainable and well-tested code
- Make sensible technical decisions and explain the trade-offs
- Prioritise effectively within a fixed timeframe
- Think about how an application would operate in a production environment
- Explain, debug and extend your own implementation

We don't expect every suggested feature to be completed, and we won't be judging the submission on feature count alone. A focused, well-built application with clear reasoning behind the decisions is preferable to a larger application that is difficult to maintain or explain.

Please focus on delivering a working solution, making sensible trade-offs and being prepared to talk us through your work.